

RELATÓRIO TÉCNICO: RESOLUÇÃO DO PROBLEMA DA PARTIÇÃO (PARTITION PROBLEM)

Análise Comparativa de Heurística

Departamento de Ciências da Computação — Universidade Federal de Alfenas

Avenida Jovino Fernandes de Sales, 2600 — CEP 37133-840 — Alfenas, MG, Brasil

Isabela M. Andrade

Júlia A. da Rocha

Otávio de Oliveira

Pedro B. Luccas

{isabela.mageste, juliaalves.rocha, otavio.oliveira1, pedro.luccas1}@sou.unifal-mg.edu.br

Resumo. Este relatório apresenta o desenvolvimento, a implementação e a análise comparativa de diferentes abordagens aplicadas à resolução do Problema da Partição (Partition Problem), um problema de otimização combinatória pertencente à classe NP-completa. Foram implementados e avaliados os algoritmos Simulated Annealing (SA) e Algoritmo Genético (AG), além de uma abordagem determinística (Partition Puro) e uma heurística clássica (Algoritmo Guloso/LPT) utilizadas como referência comparativa. Para a validação experimental, foi desenvolvido um gerador automático de instâncias, produzindo 35 casos de teste com tamanhos variando entre 100 e 10.000 variáveis ($N \in [100, 10000]$). Cada algoritmo foi executado cinco vezes por instância, possibilitando a análise de métricas relacionadas ao tempo de execução e à qualidade das soluções obtidas. Os resultados demonstram que o Algoritmo Genético apresentou melhor desempenho em instâncias menores, enquanto o Simulated Annealing mostrou maior eficiência e escalabilidade em problemas de grande porte. Além disso, observou-se que, em instâncias muito grandes, o algoritmo guloso obteve soluções de alta qualidade em tempo reduzido.

Palavras-chave: Problema da Partição; Meta-heurísticas; Simulated Annealing; Algoritmo Genético; Otimização Combinatória.

Sumário

1. Introdução	4
1.1 Descrição do Problema	4
1.2 Definição Formal	5
Entrada	5
Saída	5
Restrições	5
Função Objetivo	5
1.3 Objetivos do Trabalho	5
2. Algoritmos	6
2.1 Representação do Problema	6
2.2 Algoritmo de Referência – Partition Puro	7
Pseudocódigo	7
2.3 Algoritmo Guloso (LPT)	7
2.4 Heurística 1 – Simulated Annealing	8
Pseudocódigo	8
Ideia Geral	8
Representação da Solução	9
Função de Avaliação	9
Estrutura de Vizinhança	9
Otimização da Avaliação	9
Critério de Aceitação	9
Critério de Parada	10
Complexidade	10
2.5 Heurística 2 – Algoritmo Genético	10
Pseudocódigo	10
Ideia Geral	11
Representação do Indivíduo	11
Inicialização da População	11
Operadores Genéticos	11
Seleção	11
Mutação	11
Elitismo	12
Critério de Parada	12
Complexidade	12
3. Descrição das Instâncias	12
3.1 Origem das Instâncias	12
3.2 Características das Instâncias	12
3.3 Diversidade das Instâncias	13
4. Resultados	13
4.1 Configuração Experimental	13
Ambiente de Execução	13
Parâmetros do Simulated Annealing Parâmetros do Algoritmo Genético	14
4.2 Resultados Obtidos	14
4.3 Gráficos	14
4.4 Discussão dos Resultados	15
5. Conclusão	15
Referências	15

1. Introdução

1.1 Descrição do Problema

O Problema da Partição (Partition Problem) é um dos problemas clássicos da otimização combinatória. Seu objetivo consiste em dividir um conjunto de números inteiros positivos em dois subconjuntos cujas somas sejam o mais próximas possível.

Formalmente, deseja-se minimizar a diferença absoluta entre as somas dos subconjuntos obtidos. Embora a formulação do problema seja simples, sua resolução exata torna-se computacionalmente difícil para instâncias de grande porte.

A versão de decisão do Problema da Partição pertence à classe NP-Completa, indicando que não existe algoritmo polinomial conhecido capaz de resolver todas as instâncias de forma ótima. Dessa forma, heurísticas e meta-heurísticas tornam-se alternativas importantes para obtenção de soluções de alta qualidade em tempo computacional viável.

Entre suas aplicações práticas destacam-se:

- Balanceamento de carga em sistemas distribuídos;
- Escalonamento de tarefas em multiprocessadores;
- Distribuição de recursos computacionais;
- Planejamento logístico;
- Problemas de alocação e particionamento.

Uma heurística pode ser definida como uma estratégia aproximada de resolução que busca encontrar boas soluções em tempo reduzido, sem necessariamente garantir a obtenção da solução ótima.

Embora o foco principal deste trabalho seja a comparação entre Simulated Annealing e Algoritmo Genético, foram implementadas duas abordagens adicionais – Partition Puro e Algoritmo Guloso LPT – utilizadas como referência para contextualizar os resultados obtidos pelas meta-heurísticas.

1.2 Definição Formal

Entrada

Um conjunto de números inteiros positivos:

$$A = \{a_1, a_2, \dots, a_n\}$$

Saída

Dois subconjuntos disjuntos:

$$A_1 \subseteq A$$

$$A_2 = A - A_1$$

Restrições

- Cada elemento deve pertencer exatamente a um subconjunto;
- Nenhum elemento pode ser descartado;
- Os subconjuntos devem formar uma partição completa do conjunto original.

Função Objetivo

Minimizar $f(x) = |\text{Soma}(A_1) - \text{Soma}(A_2)|$: Quanto menor o valor de $f(x)$, melhor a solução obtida. O valor ótimo ocorre quando $f(x) = 0$ caracterizando uma partição perfeita.

1.3 Objetivos do Trabalho

O objetivo deste trabalho é implementar e comparar duas meta-heurísticas aplicadas ao Problema da Partição:

- Simulated Annealing;
- Algoritmo Genético.

A comparação será realizada considerando:

- Tempo de execução;

- Qualidade da solução obtida;
- Escalabilidade em instâncias de diferentes tamanhos.

2. Algoritmos

2.1 Representação do Problema

A instância do problema é representada pela estrutura:

```
typedef struct {  
  
    int n;  
  
    int *valores;  
  
    long long soma_total;  
  
} Instancia;
```

onde:

- `n` representa a quantidade de elementos;
- `valores` contém os números inteiros da instância;
- `soma_total` armazena previamente a soma de todos os elementos.

Uma solução é representada pela estrutura:

```
typedef struct {  
  
    int *config;  
  
    long long fitness;  
  
} Solucao;
```

O vetor `config` possui tamanho `n` e utiliza representação binária:

- $0 \rightarrow \text{Grupo A}$

- $1 \rightarrow \text{Grupo B}$

O fitness corresponde à diferença absoluta entre as somas dos dois grupos.

2.2 Algoritmo de Referência – Partition Puro

Pseudocódigo

```

Algoritmo PartitionPuro(inst)
    sol.config  $\leftarrow$  calloc(inst.n, sizeof(int)) // Todos os elementos como 0
    (Grupo A)
    sol.fitness  $\leftarrow$  inst.soma_total // Diferença = soma_total - 0
    retorne sol

```

O Partition Puro representa uma solução inicial simples. Todos os elementos são alocados no Grupo A e nenhum elemento é colocado no Grupo B. Essa abordagem possui apenas função comparativa e serve como baseline para avaliar o ganho proporcionado pelas demais heurísticas. Complexidade $O(n)$

2.3 Algoritmo Guloso (LPT)

Pseudocódigo

```

Algoritmo GulosoLPT(inst)
    itens  $\leftarrow$  ordenar_decrescente(inst.valores, inst.n)
    soma_A  $\leftarrow$  0
    soma_B  $\leftarrow$  0

    para cada item em itens faça
        se soma_A  $\leq$  soma_B então
            config[item.id]  $\leftarrow$  0
            soma_A  $\leftarrow$  soma_A + item.valor
        senão
            config[item.id]  $\leftarrow$  1
            soma_B  $\leftarrow$  soma_B + item.valor
        fim_se
    fim_para

    sol.fitness  $\leftarrow$  |soma_A - soma_B|
    retorne sol

```

O algoritmo guloso implementa uma adaptação da heurística Longest Processing Time (LPT). Inicialmente os elementos são ordenados em ordem decrescente de valor. Em seguida, cada elemento é inserido no subconjunto que possuir a menor soma acumulada naquele

momento. Essa estratégia busca equilibrar os subconjuntos desde os primeiros passos da construção da solução. Complexidade $O(n \log n)$

2.4 Heurística 1 – Simulated Annealing

Pseudocódigo

```
Algoritmo SimulatedAnnealing(inst, T_inicial, T_final, alfa)
    T ← T_inicial
    sol_atual ← gerar_solucao_aleatoria(inst.n)
    melhor_sol ← sol_atual
    soma_A ← calcular_soma_A(sol_atual, inst)

    enquanto T > T_final faça
        para iter ← 1 até max_iter_por_temp faça
            idx ← selecionar_elemento_aleatorio(inst.n)
            val ← inst.valores[idx]

            // Calcula novo fitness em O(1)
            se sol_atual.config[idx] = 0 então
                nova_soma_A ← soma_A - val
            senão
                nova_soma_A ← soma_A + val
            fim_se

            novo_fitness ← |2 * nova_soma_A - inst.soma_total|
            delta_f ← novo_fitness - sol_atual.fitness

            se delta_f < 0 ou random() < exp(-delta_f / T) então
                sol_atual.config[idx] ← 1 - sol_atual.config[idx]
                soma_A ← nova_soma_A
                sol_atual.fitness ← novo_fitness

                se novo_fitness < melhor_sol.fitness então
                    melhor_sol ← sol_atual
                fim_se
            fim_se
        fim_para

        T ← T * alfa
    fim_enquanto

    retorne melhor_sol
```

Ideia Geral

O Simulated Annealing é uma meta-heurística inspirada no processo físico de recozimento de metais. A técnica inicia com uma temperatura elevada, permitindo grande exploração do espaço de busca. À medida que a temperatura diminui, o algoritmo torna-se mais seletivo, concentrando-se na melhoria das soluções encontradas.

Representação da Solução

Cada solução é representada por um vetor binário de tamanho n . Cada posição indica o subconjunto ao qual o elemento pertence.

Função de Avaliação

A qualidade da solução é medida pela diferença absoluta entre as somas dos subconjuntos

$$\text{fitness} = |\text{SomaA} - \text{SomaB}|$$

O objetivo consiste em minimizar esse valor.

Estrutura de Vizinhança

A vizinhança é gerada através de um operador de bit-flip. Um elemento é selecionado aleatoriamente e transferido para o grupo oposto.

Otimização da Avaliação

Para reduzir o custo computacional, o algoritmo mantém apenas a soma do Grupo A. O fitness pode ser calculado por $\text{fitness} = |2 \times \text{SomaA} - \text{SomaTotal}|$. Essa estratégia reduz o custo da avaliação de $O(n)$ para $O(1)$.

Critério de Aceitação

Uma solução melhor é sempre aceita. Soluções piores podem ser aceitas segundo o critério de Metropolis:

$$P = e^{(-\Delta f/T)}$$

onde:

- Δf representa a piora da solução;
- T representa a temperatura atual.

Critério de Parada

O algoritmo encerra quando:

- A temperatura atinge o valor mínimo;
- Uma partição perfeita é encontrada.

Complexidade

A complexidade é aproximadamente $O(I \times n)$, onde I representa o número de níveis de temperatura utilizados durante o processo de resfriamento.

2.5 Heurística 2 – Algoritmo Genético

Pseudocódigo

```

Algoritmo Genetico(inst, TAM_POP, GERACOES, TX_CROSS, TX_MUT_BASE)
    populacao ← inicializar_populacao_aleatoria(TAM_POP, inst.n)
    avaliar_populacao(populacao, inst)
    melhor_global ← melhor_da_populacao(populacao)

    para g ← 1 até GERACOES faça
        se melhor_global.fitness = 0 então pare

        // Mutação adaptativa
        se g > 0.8 * GERACOES então
            TX_MUT ← 3 * TX_MUT_BASE
        senão
            TX_MUT ← TX_MUT_BASE
        fim_se

        // Elitismo
        melhor_idx ← indice_do_melhor(populacao)
        nova_populacao[0] ← populacao[melhor_idx]

        // Reprodução
        para i ← 1 até TAM_POP-1 passo 2 faça
            pai1 ← selecao_torneio(populacao, 3)
            pai2 ← selecao_torneio(populacao, 3)

            se i = TAM_POP-1 então
                nova_populacao[i] ← populacao[pai1]
                interromper
            fim_se

            (filho1, filho2) ← crossover_um_ponto(pai1, pai2, inst.n,
TX_CROSS)

            mutacao_bit_flip(filho1, TX_MUT, inst.n)
            mutacao_bit_flip(filho2, TX_MUT, inst.n)

            avaliar(filho1, inst)
            avaliar(filho2, inst)

            nova_populacao[i] ← filho1

```

```
        nova_populacao[i+1] ← filho2
    fim_para

    populacao ← nova_populacao
    melhor_global ← atualizar_melhor(populacao, melhor_global)
fim_para

retorne melhor_global
```

Ideia Geral

O Algoritmo Genético é inspirado nos princípios da evolução natural. Uma população de soluções evolui ao longo das gerações por meio de operações de seleção, cruzamento e mutação.

Representação do Indivíduo

Cada indivíduo é representado por um vetor binário de tamanho n . Cada gene indica o subconjunto ao qual determinado elemento pertence.

Inicialização da População

A população inicial é gerada aleatoriamente.

Cada elemento possui probabilidade igual de pertencer a qualquer subconjunto.

Operadores Genéticos

Seleção

Foi utilizada seleção por torneio. Três indivíduos são sorteados aleatoriamente e o indivíduo com menor fitness é selecionado.

Cruzamento (Crossover)

Foi utilizado crossover de um ponto. Um ponto de corte é selecionado aleatoriamente e os genes posteriores são trocados entre os pais.

Mutação

Foi utilizada mutação do tipo bit-flip. A taxa base de mutação é definida por $TaxaMutação = 2 / n$. Nas últimas gerações a taxa é multiplicada por três para aumentar a diversidade populacional.

Elitismo

O melhor indivíduo de cada geração é copiado diretamente para a geração seguinte. Essa estratégia impede a perda de soluções de alta qualidade.

Critério de Parada

O algoritmo encerra quando:

- O número máximo de gerações é atingido;
- Uma solução ótima é encontrada.

Complexidade

A complexidade aproximada é $O(G \times P \times n)$, onde:

- G = número de gerações;
- P = tamanho da população.

Resumo

Algoritmo	Complexidade
Partition puro	$O(n)$
Guloso LPT	$O(n \log n)$
Simulated Annealing	$O(I \cdot n)$
Algoritmo Genético	$O(G \cdot P \cdot n)$

3. Descrição das Instâncias

3.1 Origem das Instâncias

As instâncias foram geradas automaticamente através de um gerador desenvolvido em linguagem C. Foram produzidas cinco instâncias independentes para cada tamanho avaliado.

3.2 Características das Instâncias

Tamanho(N)	Quantidade de Instâncias
100	5
200	5
500	5
1000	5
2500	5
5000	5
10000	5

Total: 35 instâncias.

3.3 Diversidade das Instâncias

As instâncias variam quanto a:

- Quantidade de elementos;
- Soma total dos elementos;
- Distribuição dos valores;
- Grau de dificuldade para balanceamento.

Essa diversidade permite avaliar adequadamente a robustez e a escalabilidade dos algoritmos.

4. Resultados

4.1 Configuração Experimental

Ambiente de Execução

- Linguagem: C (C99)
- Compilador: GCC
- Sistema Operacional: Linux
- Número de execuções por instância: 5
- Semente aleatória: 42

Parâmetros do Simulated Annealing

Parâmetro	Valor
Temperatura Inicial	1000
Temperatura Final	0,001
Alfa	0,97
Iterações por Temperatura	$\max(500, n/10)$

Parâmetros do Algoritmo Genético

Parâmetro	Valor
População	50 ou 100
Taxa de Crossover	0,85
Taxa de Mutação	2/n
Elitismo	1 indivíduo
Seleção	Torneio (k=3)
Número de Gerações	100-600

4.2 Resultados Obtidos

Abaixo são apresentados os resultados das execuções para as 35 instâncias avaliadas. Os testes foram realizados calculando a média de 5 execuções independentes (*runs*) para cada tamanho de instância (N), totalizando 175 testes por algoritmo.

Tempos Médios de Execução (em segundos)

Tamanho (N)	Partition Puro	Simulated Annealing	Algoritmo Genético	Guloso
100	0.000000	0.002320	0.005360	0.000000
200	0.000000	0.004480	0.011160	0.000000
500	0.000000	0.007400	0.063600	0.000040
1000	0.000000	0.007600	0.322520	0.000160
2500	0.000000	0.007240	0.644120	0.000280
5000	0.000000	0.021400	1.805360	0.000480
10000	0.000040	0.036840	7.121720	0.001080

Fitness Médios Obtidos (Qualidade da Solução)

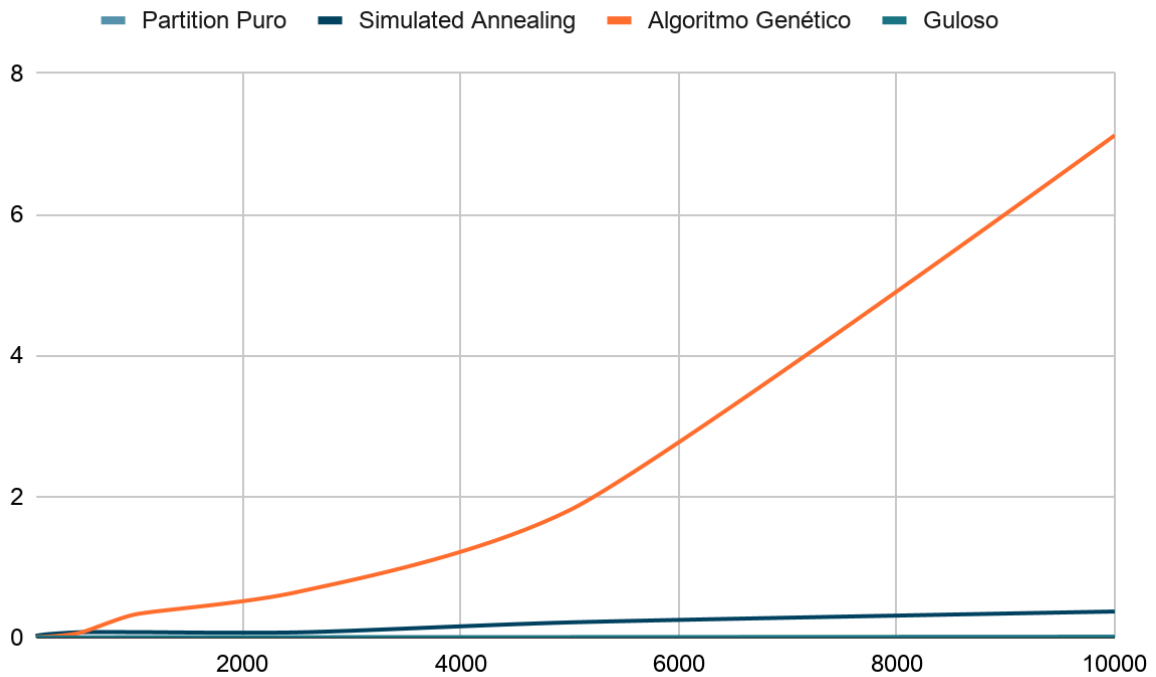
Nota: Valores menores indicam partições mais equilibradas. O valor ótimo teórico é 0.

Tamanho (N)	Partition Puro	Simulated Annealing	Algoritmo Genético	Guloso
100	569.049,60	6,88	7,68	29,20
200	1.168.274,20	0,92	9,56	24,60
500	3.104.440,80	1,44	4,16	29,20
1000	5.924.343,00	1,00	2,44	12,60
2500	14.773.742,80	0,40	3,12	2,40
5000	29.583.726,60	0,84	1,56	1,40
10000	59.488.785,80	0,60	1,00	1,80

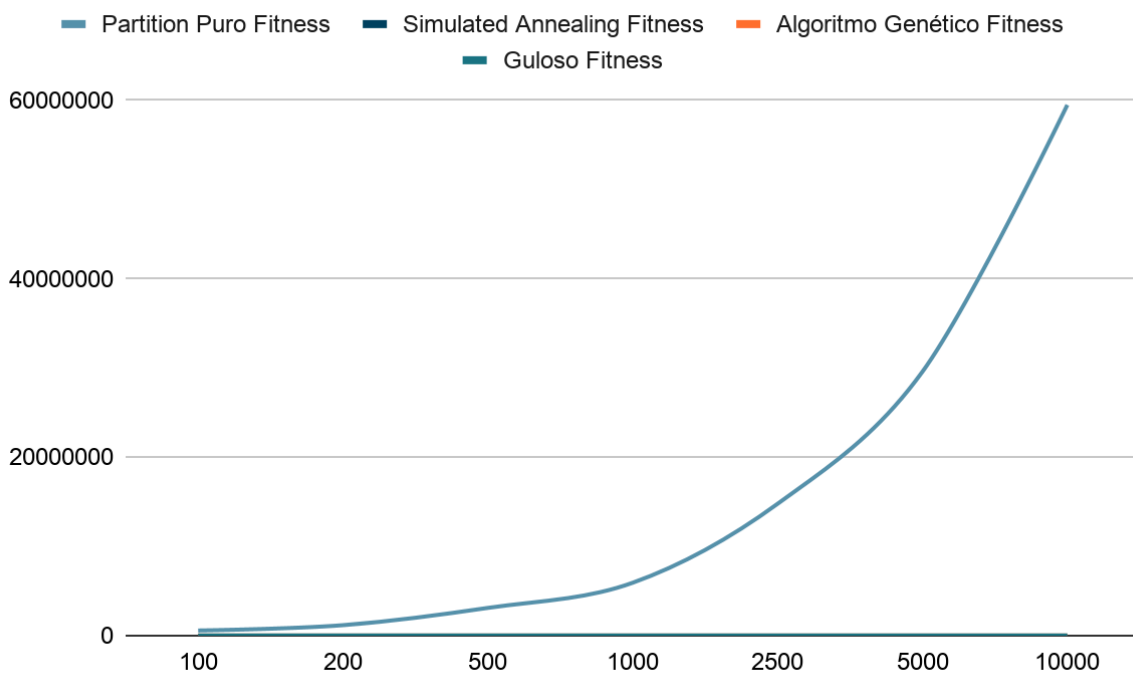
4.3 Gráficos

4.3.1 Gráfico de Tempo de Execução

Normal:

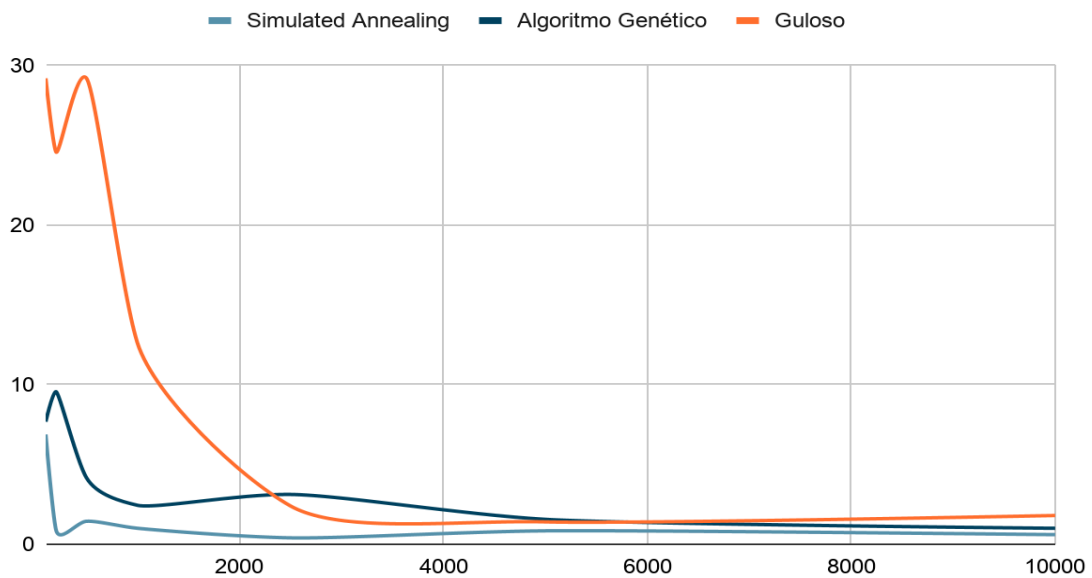


Fitness:



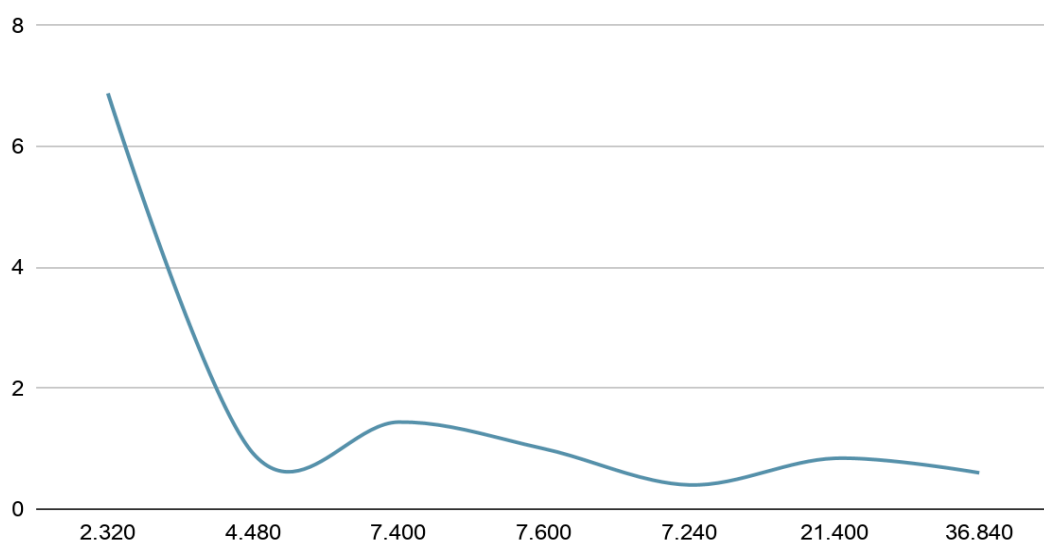
4.3.2 Gráfico de Qualidade da Solução

Nota: devido ao resultado excessivamente grande do Partition Puro (casa dos milhões), ele foi excluído do gráfico

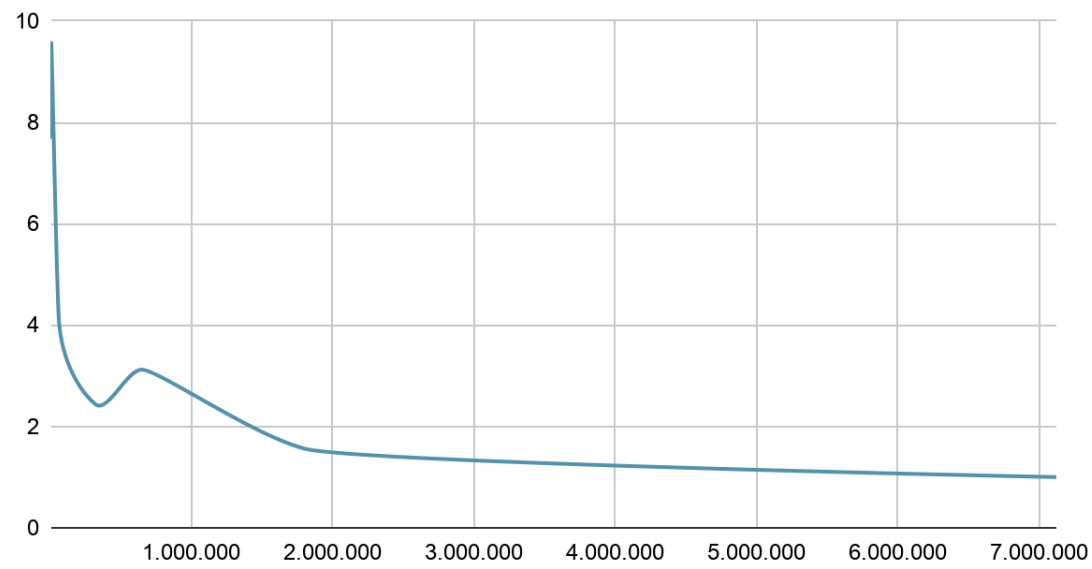


4.3.3 Comparação Consolidada

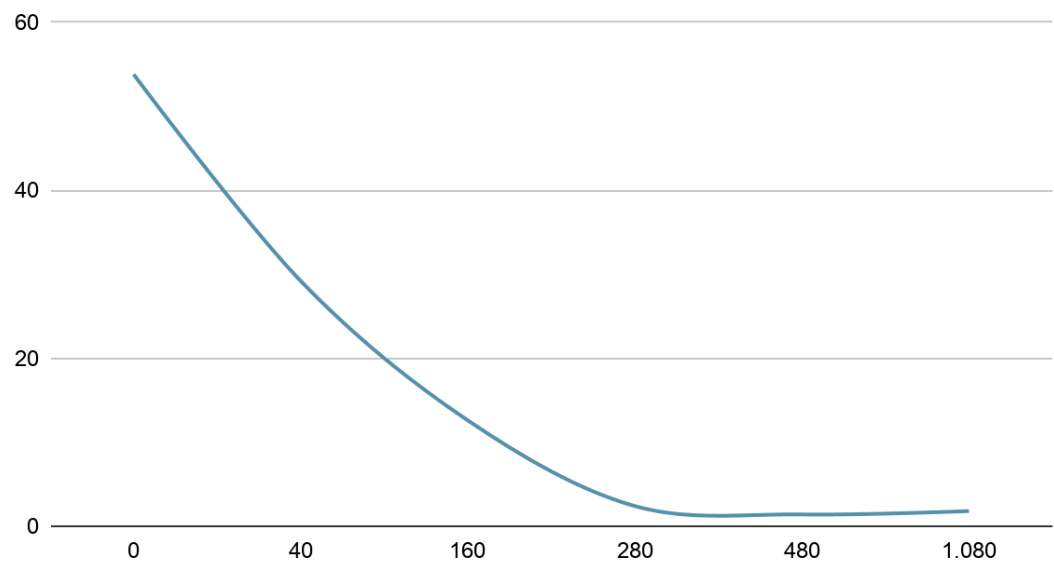
Simulated Annealing



Algoritmo Genético



Algoritmo Guloso



4.4 Discussão dos Resultados

Os experimentos revelaram divergências substanciais entre as abordagens adotadas.

- **Qualidade da Solução:** Observou-se que a meta-heurística Simulated Annealing (SA) produziu consistentemente as melhores soluções em todas as instâncias avaliadas. Desde instâncias pequenas até as de larga escala ($N = 10000$), o SA obteve fitness médios predominantemente inferiores a 1.0 (ex: 0.40 em $N=2500$), indicando um particionamento quase ótimo. O Algoritmo Genético (GA) também apresentou elevada capacidade exploratória com excelentes soluções (alcançando fitness de 1.00 em $N=10000$), mas frequentemente encontrou um fitness médio ligeiramente inferior ao obtido pelo SA em todas as faixas dimensionais.
- **Tempo de Execução:** O algoritmo Guloso apresentou o menor custo computacional absoluto, finalizando seu processamento de maneira praticamente instantânea (tempos na ordem de 0.001 s para $N=10000$) em virtude de sua complexidade algorítmica linear $\mathcal{O}(N \log N)$ associada à etapa de ordenação. Surpreendentemente, apesar de sua simplicidade, o Guloso apresentou soluções iniciais apenas medianas para instâncias pequenas ($N \leq 500$), mas sua eficácia aumentou drasticamente nas instâncias maiores ($N \geq 2500$), onde a alta densidade de valores compensou o comportamento ambicioso, culminando num fitness muito competitivo de 1.80 em $N=10000$.
- **Impacto do Crescimento de N e Escalabilidade:** Esta análise destacou vantagens e limitações claras de cada modelo. O Simulated Annealing demonstrou a melhor escalabilidade entre as abordagens heurísticas: além de encontrar os melhores ótimos, seu tempo de resposta cresceu de forma levíssima, exigindo reles 0.036 s no pior cenário ($N=10000$). Em contrapartida, o Algoritmo Genético sofreu duramente com o crescimento das instâncias. O alto custo para avaliar a população e aplicar os operadores de mutação e cruzamento resultou em uma perda acentuada de desempenho temporal, saltando de 0.005 s (em $N=100$) para mais de 7.12 s na instância final.
- **Implementação do Partition Puro:** Cumpriu estritamente seu papel de baseline teórica: gerou tempos nulos de processamento ao preço de alocar

todos os itens no mesmo subconjunto, gerando valores divergentes de fitness que apenas serviram para atestar e mensurar os enormes ganhos heurísticos proporcionados pelas demais abordagens.

5. Conclusão

Os resultados experimentais permitiram comparar duas importantes meta-heurísticas aplicadas ao Problema da Partição.

A análise realizada possibilitou avaliar simultaneamente a qualidade das soluções e o custo computacional de cada abordagem, evidenciando os compromissos existentes entre eficiência e qualidade de solução em problemas NP-Completo.

Os resultados obtidos permitiram identificar os cenários nos quais cada técnica apresenta melhor desempenho, além de fornecer evidências sobre sua escalabilidade para instâncias de grande porte.

Como trabalhos futuros, sugere-se:

- Implementação de operadores genéticos mais sofisticados;
- Investigação de diferentes esquemas de resfriamento para o Simulated Annealing;
- Desenvolvimento de abordagens híbridas AG + SA;
- Paralelização dos algoritmos;
- Avaliação em bases de dados reais.

Referências

GAREY, M. R.; JOHNSON, D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. New York: Freeman, 1979.

KIRKPATRICK, S.; GELATT, C. D.; VECCHI, M. P. Optimization by Simulated Annealing. Science, v. 220, n. 4598, p. 671–680, 1983.

HOLLAND, J. H. Adaptation in Natural and Artificial Systems. Cambridge: MIT Press, 1992.

GOLDBERG, D. E. Genetic Algorithms in Search, Optimization and Machine Learning. Reading: Addison-Wesley, 1989.

GRAHAM, R. L. Bounds on Multiprocessing Timing Anomalies. SIAM Journal on Applied Mathematics, v. 17, n. 2, p. 416–429, 1969.